# Towards Sustainable Software Release Management using Incremental Builds and Carbon-Aware Scheduling

R26-SE-027

Project Proposal Report

Mendis J.D.L – IT22110084

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

## DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

| Name | Student ID | Signature |
|---|---|---|
| Mendis J.D.L. | **IT22110084** |  |

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

| Name | Date | Signature |
|---|---|---|
| Mr.Jagath Wickramarathna | 2026.03.11 |  |

**ABSTRACT**

Continuous Integration and Continuous Deployment (CI/CD) pipelines are essential for modern software delivery but are highly resource-intensive, contributing significantly to the growing carbon footprint of the Information and Communication Technology (ICT) sector. Traditional CI/CD setups often execute redundant full-system builds and exhaustive test suites regardless of the change scope, drawing power without considering the carbon intensity of the underlying electricity grid. While major cloud providers are beginning to explore carbon-aware computing, existing CI/CD orchestrators lack an automated, intelligent decision-making layer capable of profiling deployment energy and scheduling non-urgent tasks during low-carbon windows without sacrificing delivery performance.

To bridge this gap, this research proposes an AI-driven sustainable release management framework. The proposed solution introduces a dynamic decision layer featuring a Deployment Energy Profiler and an AI Decision Engine. The framework bypasses redundant computational workloads through change impact analysis, artifact reuse, and simplified risk-based test selection. For non-urgent tasks, an XGBoost supervised machine learning model evaluates real-time and forecasted grid carbon data via external APIs (e.g., WattTime) to predict and schedule optimal, low-carbon deployment windows. The system will be implemented using Python, Docker, and Jenkins/GitHub Actions, with REST APIs facilitating communication between the CI/CD orchestrator and the AI engine.

The prototype will be experimentally evaluated against a traditional baseline pipeline. Expected outcomes include a demonstrable reduction in pipeline execution time, lower CPU/memory utilization, and decreased estimated energy consumption. Commercially, this framework offers enterprises significant reductions in cloud infrastructure costs and optimized CI/CD resource utilization. Socially, it directly supports organizational Environmental, Social, and Governance (ESG) initiatives by actively reducing the greenhouse gas emissions associated with software engineering.

## ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, **Mr. Jagath Wickramarathna**, for his valuable guidance, constructive feedback, and continuous support throughout the preparation of this research proposal. His expertise and encouragement greatly contributed to shaping the direction and quality of this work.

I would also like to extend my appreciation to my co-supervisor, **Ms. Madusha Weerasooriya**, for her insightful suggestions and support during the development of this research proposal.

My sincere thanks also go to our external supervisor, **Mr. Dilanka Nayanajith**, for his industry insights, professional advice, and valuable guidance which helped us better understand the practical aspects of the research.

Furthermore, I would like to thank the academic staff of the **Sri Lanka Institute of Information Technology (SLIIT)** for providing the necessary knowledge and resources that supported the preparation of this proposal.

Finally, I would like to express my appreciation to my fellow team members **Mendis J.D.L.**, **Thassara M.P.M.**, and **Kodithuwakku I.P.** for their collaboration, discussions, and teamwork throughout the early stages of this research.

I am also grateful to everyone who contributed directly or indirectly to the successful preparation of this research proposal.

# Table of Contents

**TOWARDS SUSTAINABLE SOFTWARE RELEASE MANAGEMENT USING INCREMENTAL BUILDS AND CARBON-AWARE SCHEDULING ..... 1**

*DECLARATION ..... 2*
*ABSTRACT ..... 3*
*ACKNOWLEDGEMENT ..... 4*

**LIST OF TABLES ..... 7**

*LIST OF FIGURES ..... 8*
*LIST OF ABBREVIATIONS ..... 9*
*LIST OF APPENDICES ..... 10*

**1. INTRODUCTION ..... 11**

1.1. Background and Context ..... 11
1.2. Literature Review ..... 12
1.3. Research Gap ..... 13

**2. OBJECTIVE ..... 14**

2.1. Main objective ..... 14
2.2. Specific Objectives ..... 14
*Objective 1 — Design a Sustainable Release Management Architecture ..... 14*
*Objective 2 — Implement Artifact Reuse and Incremental Build Mechanisms ..... 14*
*Objective 3 — Develop a Selective Release Verification Strategy ..... 14*
*Objective 4 — Propose a Carbon-Aware Release Scheduling Model ..... 14*
*Objective 5 — Evaluate the Effectiveness of the Proposed Framework ..... 15*

**3. METHODOLOGY ..... 16**

3.1. Research Approach ..... 16
3.2. System Architecture Overview ..... 16
3.3. Technical Methods and Algorithms ..... 17
*3.3.1. Change Impact Analysis ..... 17*
*3.3.2. Artifact Reuse Strategy ..... 17*
*3.3.3. Selective Test Execution(Simplified Risk-Based Testing) ..... 17*
*3.3.4. AI-Driven Carbon-Aware Scheduling ..... 18*

**4. HIGH-LEVEL SYSTEM ARCHITECTURE ..... 19**

4.1. System Overview ..... 19
4.2. System Components and interaction ..... 19
4.3. Data and Execution Flow ..... 20

**5. USER REQUIREMENTS ..... 23**

5.1. Stakeholder Identification ..... 23
5.2. Functional Requirements ..... 23
5.3. Non-Functional Requirements ..... 24

**6. COMMERCIALIZATION PLAN ..... 25**

6.1. Target Market and Customer Persona ..... 25
6.2. Value Proposition ..... 25
*6.3. Revenue Model and Pricing Strategy ..... 25*

6.4. COMPETITIVE ADVANTAGE ..... 26

**7. BUDGET AND JUSTIFICATION ..... 26**

**WORK BREAKDOWN STRUCTURE (WBS) ..... 27**

*9. GANTT CHART ..... 29*

*10. REFERENCES LIST ..... 30*

*11. APPENDICES ..... 32*

# List Of tables

Table 1 Proposed SaaS Revenue Model and Pricing Strategy ..... 25
Table 2 Estimated Project Budget and Cost Justification ..... 26
Table 3 Work Breakdown Structure ..... 27

## LIST OF FIGURES

Figure 1 Overall System Diagram ..... 21
Figure 2 System Diagram of components..... 22

## LIST OF ABBREVIATIONS

| Abbreviation | Definition / Full Form |
|---|---|
| **AI** | Artificial Intelligence |
| **API** | Application Programming Interface |
| **AWS** | Amazon Web Services |
| **CD** | Continuous Delivery / Continuous Deployment |
| **CI** | Continuous Integration |
| **CPU** | Central Processing Unit |
| **CSI** | Container Storage Interface |
| **CT** | Continuous Testing |
| **DAG** | Directed Acyclic Graph |
| **DAST** | Dynamic Application Security Testing |
| **DSRM** | Design Science Research Methodology |
| **ESG** | Environmental, Social, and Governance |
| **GCP** | Google Cloud Platform |
| **gCO2/kWh** | Grams of Carbon Dioxide per Kilowatt-hour |
| **GHG** | Greenhouse Gas |
| **GUI** | Graphical User Interface |
| **HTTP** | Hypertext Transfer Protocol |
| **IAM** | Identity and Access Management |
| **ICT** | Information and Communication Technology |
| **IDE** | Integrated Development Environment |
| **IoT** | Internet of Things |
| **JSON** | JavaScript Object Notation |
| **JWT** | JSON Web Token |
| **LKR** | Sri Lankan Rupee |
| **ML** | Machine Learning |
| **OS** | Operating System |
| **PII** | Personally Identifiable Information |
| **PUE** | Power Usage Effectiveness |
| **PV** | Persistent Volume |
| **PVC** | Persistent Volume Claim |
| **RAM** | Random Access Memory |
| **RAPL** | Running Average Power Limit |
| **RBAC** | Role-Based Access Control |
| **REST** | Representational State Transfer |
| **ROI** | Return on Investment |
| **SaaS** | Software as a Service |
| **SAST** | Static Application Security Testing |
| **SCA** | Software Composition Analysis |
| **SDS** | Software Defined Storage |
| **SUT** | System Under Test |
| **TaaS** | Testing as a Service |
| **TDD** | Test-Driven Development |
| **UI** | User Interface |
| **VCS** | Version Control System |
| **VGT** | Visual GUI Testing |
| **VM** | Virtual Machine |

| | |
|---|---|
| **WBS** | Work Breakdown Structure |

## LIST OF APPENDICES

Appendice 1Risk Analysis and Mitigation plan ..... 33
Appendice 2 Detailed Data flow diagram ..... 34
Appendice 3 Detailed API Interaction diagram ..... 35

# 1. Introduction

## 1.1. Background and Context

The rapid evolution of software development practices has made Continuous Integration (CI) and Continuous Deployment (CD) pipelines an essential component of modern software engineering [1]. In contemporary cloud-native deployments, organizations increasingly standardize delivery via highly automated CI/CD and GitOps processes, with 38% of organizations reporting that 80–100% of their releases are fully automated [2]. However, this relentless pursuit of speed and automation has a significant environmental cost. The Information and Communication Technology (ICT) sector currently accounts for 5%–9% of global electricity consumption and over 2% of greenhouse gas emissions, while data centers alone consume approximately 1-2% of global electricity. [3] [4]

A major contributor to this energy consumption is the redundant computational work performed during automated software releases. Traditional CI/CD pipelines often execute full system rebuilds and run exhaustive test suites even when only minor changes are committed to the repository [5]. Studies estimate that nearly 90% of CPU time on CI workers can be wasted when full rebuilds are executed for minor changes [6]. This inefficiency affects software organizations by increasing cloud infrastructure costs and lengthening developer feedback cycles, while simultaneously affecting the global environment through unnecessary carbon emissions. [7]

Existing solutions to this problem, such as build caching and incremental compilation, are significantly underutilized in CI/CD environments. For instance, an empirical study of GitHub projects revealed that 70% of projects do not adopt CI caching, primarily due to a lack of awareness or misconfigurations [8]. Furthermore, current CI/CD architectures prioritize throughput and latency, typically ignoring the carbon intensity (gCO2/kWh) of the electricity powering the underlying infrastructure [9].

To address these inefficiencies, this research proposes a sustainable release orchestration framework that utilizes three distinct sustainability levers: (1) Avoiding Unnecessary Release Work via artifact reuse and incremental builds, (2) Smarter Release Verification via rule-based selective testing, and (3) Carbon-Aware Release Scheduling to align non-urgent pipeline executions with periods of low grid carbon intensity [3] [9]. This research aligns directly with the United Nations Sustainable Development Goals (SDGs), specifically SDG 9 (Industry, Innovation, and Infrastructure), SDG 12 (Responsible Consumption and Production), and SDG 13 (Climate Action), and fits within the Green Computing and Software Engineering research clusters.

## 1.2. Literature Review

The environmental impact of CI/CD pipelines and methods for optimizing software builds have garnered increasing attention in recent academic literature.

*Incremental Builds and Build Caching:* Maudoux and Mens highlight that CI environments traditionally shun incremental builds in favor of clean builds to guarantee correctness, which inadvertently wastes massive amounts of computational resources [6]. Tiinanen demonstrated that introducing persistent workspace caching in distributed CI environments can drastically reduce source code cloning and compilation times [10]. However, Ghaleb et al. found that despite the availability of caching in platforms like Travis CI, only 30% of projects adopt it, and among those that do, poor maintenance often leads to caching redundant artifacts like installation logs, which actually increases build overhead [10]. Curtsinger and Barowy further emphasize that traditional build tools require labor-intensive manual dependency configurations to achieve incremental builds, proposing forward-build tools to automatically capture dependencies and reduce redundant work [5].

*Smarter Release Verification:* Time-consuming testing is repeatedly cited as a primary bottleneck in continuous delivery [11]. Mascheroni and Irrazábal conducted a systematic literature review on Continuous Testing, identifying that executing massive automated test suites for every minor change is a pervasive problem [11]. To mitigate this, researchers propose test case selection and prioritization techniques that execute only the tests correlated to the modified source code, thus saving time and computational effort without sacrificing quality [11].

*Energy-Aware CI/CD and Carbon Scheduling:* The integration of energy metrics into CI/CD is an emerging field. The PPTAMη framework was recently introduced as an energy-aware CI/CD pipeline capable of measuring CPU and DRAM power consumption per commit, making the energy footprint of software changes visible to developers [12]. Beyond simply measuring energy, actual carbon-aware resource allocation proposes shifting compute loads to align with renewable energy availability. Pinnapareddy emphasizes that flexible workloads, such as batch processing or CI builds, can be delayed safely to coincide with low-carbon energy windows using predictive data from APIs like ElectricityMap or WattTime [9]. Microsoft's implementation of Carbon-Aware Kubernetes has demonstrated that such temporal and spatial workload shifting can yield up to a 34% reduction in carbon emissions without negatively impacting critical service level objectives [7]. Finally, Peltonen et al. advocate for incorporating Green Software Foundation principles directly into the application and middleware layers, ensuring that software development processes natively consider carbon efficiency [4].

## 1.3. Research Gap

While the literature provides substantial evidence for the benefits of incremental builds, test selection, and carbon-aware computing individually, there is a distinct lack of research that holistically integrates all three concepts into a unified CI/CD release management framework. Currently, CI/CD optimization approaches are highly fragmented. Studies on build caching focus almost entirely on developer feedback speed rather than energy reduction [10]. Conversely, research on carbon-aware computing typically focuses on large-scale data center orchestration or container scheduling rather than the software delivery pipeline itself [9]. Furthermore, tools that do monitor energy in CI/CD pipelines (such as PPTAMη) are primarily observational and diagnostic, lacking the active orchestration mechanisms required to automatically prevent redundant work or reschedule tasks based on grid carbon intensity [12].

This research addresses this gap by proposing and evaluating a multi-lever sustainable release orchestration framework. The novelty of this approach lies in its layered strategy: it first minimizes the absolute volume of computational work through dependency-aware incremental builds and rule-based test selection, and then applies a conceptual carbon-aware scheduling layer to the remaining necessary work. By evaluating this optimized pipeline against a traditional baseline pipeline, this study will provide vital empirical evidence on the intersection of release management efficiency and software sustainability, offering a practical blueprint for reducing the carbon footprint of modern software engineering practices.

# 2. OBJECTIVE

## 2.1. Main objective

To design, implement, and evaluate a sustainable release orchestration framework for CI/CD pipelines that significantly reduces redundant computational workload and energy consumption by integrating incremental builds, rule-based test selection, and carbon-aware scheduling, while maintaining software delivery speed and quality.

## 2.2. Specific Objectives

To achieve the main objective, the following specific objectives are defined:

**Objective 1 — Design a Sustainable Release Management Architecture**

Design a CI/CD pipeline architecture that integrates sustainable release management principles, including artifact reuse mechanisms, incremental build strategies, selective verification techniques, and carbon-aware scheduling concepts.

This objective focuses on defining the **overall framework architecture and decision logic** required to determine when builds, tests, or deployments should be executed.

---

**Objective 2 — Implement Artifact Reuse and Incremental Build Mechanisms**

Develop and integrate artifact reuse and incremental build mechanisms within a CI/CD pipeline to avoid rebuilding software components that have not been modified.

This includes implementing **change impact analysis**, module-level rebuild detection, and artifact repository integration to ensure that previously generated build artifacts can be reused safely during release processes.

---

**Objective 3 — Develop a Selective Release Verification Strategy**

Design and implement a simplified **risk-based test selection mechanism** that prioritizes critical tests while reducing the execution of unnecessary test cases during release verification.

The objective is to maintain software quality while reducing the computational intensity of testing stages within CI/CD pipelines.

---

**Objective 4 — Propose a Carbon-Aware Release Scheduling Model**

Develop a conceptual carbon-aware scheduling framework that aligns non-urgent CI/CD release tasks with periods of lower carbon intensity in electricity grids.

This objective will explore how external carbon intensity data can inform release scheduling decisions in future sustainable DevOps environments.

---

**Objective 5 — Evaluate the Effectiveness of the Proposed Framework**

Experimentally evaluate the proposed sustainable release management framework by comparing it with traditional CI/CD release pipelines using metrics such as:

- pipeline execution time
- number of rebuilt modules
- CPU and computational workload
- estimated energy consumption

The evaluation will determine whether the proposed framework can reduce unnecessary computation while maintaining release reliability and software quality.

# 3. METHODOLOGY

## 3.1. Research Approach

This research adopts a Design Science Research (DSR) approach combined with experimental evaluation and machine learning modeling. Design Science is highly appropriate because the study focuses on designing and evaluating a technical artifact—an AI-driven, sustainable release management framework for CI/CD pipelines. The research will develop a prototype CI/CD pipeline architecture that integrates multiple sustainability mechanisms, highlighted by a machine-learning-based decision engine, and evaluate its effectiveness through controlled experiments.

The study will proceed through the following stages:

1. **Problem Identification:** Analyze inefficiencies in conventional CI/CD release pipelines, particularly redundant builds, repeated test executions, and the carbon impact of scheduling deployments blindly during peak grid-carbon hours.
2. **Artifact Design:** Design a sustainable release orchestration framework based on three core sustainability levers: avoiding unnecessary release work (artifact reuse), smarter release verification (selective testing), and intelligent scheduling (AI-driven carbon-aware deployment).
3. **Model Development & Prototype Implementation:** Develop a Deployment Energy Profiler to gather metrics, train an XGBoost supervised learning model to predict optimal scheduling, and implement the cohesive framework within a CI/CD environment using modern DevOps tools.
4. **Experimental Evaluation:** Compare the optimized, AI-driven pipeline with a baseline traditional pipeline to measure improvements in execution time, computational workload, and estimated energy consumption.

## 3.2. System Architecture Overview

The proposed sustainable release framework introduces an intelligent decision layer within the CI/CD pipeline that determines *what* tasks should be executed and *when* they should run. The architecture consists of the following components:

1. **Change Impact Analyzer & Artifact Reuse Engine:** Detects which modules have been modified and determines whether previously generated build artifacts can be fetched from a repository (e.g., Nexus) instead of rebuilding the system.
2. **Incremental Build Module:** Compiles only the affected modules when source code changes are detected.
3. **Selective Test Execution Engine:** Executes only prioritized or high-risk test cases based on predefined rules regarding the release type.

4. **Deployment Energy Profiler:** A Python-based monitoring component that extracts runtime metrics (CPU usage, memory usage, and deployment duration) from CI/CD server logs and system resource monitors.
5. **AI Decision Engine (XGBoost):** Evaluates real-time and forecasted carbon intensity data against profiled deployment metrics to identify optimal scheduling windows for non-urgent release tasks.

## 3.3. Technical Methods and Algorithms

### 3.3.1. Change Impact Analysis

The first step in the release pipeline is identifying the **scope of changes** introduced in a new commit. A change impact analysis algorithm will analyze modified files and map them to corresponding system modules.

Example logic:

```
For each commit:
    Identify changed files
    Map changed files to project modules
    Determine dependent modules
    Mark modules requiring rebuild
```

This process ensures that only the necessary components are rebuilt.

### 3.3.2. Artifact Reuse Strategy

To avoid unnecessary rebuilds, the system will use an artifact repository to store compiled outputs such as build artifacts. When a release is triggered, the pipeline checks whether a valid artifact already exists.

Decision rule:

```
IF artifact exists AND module unchanged
    Reuse artifact
ELSE
    Rebuild module
```

Artifact reuse reduces redundant compilation tasks and significantly decreases release pipeline workload.

### 3.3.3. Selective Test Execution(Simplified Risk-Based Testing)

Instead of executing the complete test suite for every release, the pipeline will classify tests into categories:

| Test Category | Purpose |
|---|---|
| **Critical tests** | Validate core system functionality |
| **Integration tests** | Verify component interactions |

| | |
|---|---|
| **Extended tests** | Perform comprehensive validation |

The release pipeline executes test suites depending on release type:

| Release Type | Tests Executed |
|---|---|
| **Patch release** | Critical tests |
| **Minor release** | Critical + Integration tests |
| **Major release** | Full test suite |

This approach reduces computational overhead while maintaining acceptable test coverage.

#### 3.3.4. AI-Driven Carbon-Aware Scheduling

For non-urgent deployments, the framework utilizes a machine learning approach to delay execution until the electricity grid is powered by low-carbon energy sources. This is executed in three distinct phases:

- **Phase 1: Data Extraction and Profiling:** The Deployment Energy Profiler utilizes Python scripts to continuously extract historical deployment durations and system status from CI/CD logs. Simultaneously, it fetches external grid carbon intensity data (measured in gCO2/kWh) using APIs such as WattTime.
- **Phase 2: Data Preprocessing:** The extracted data is aggregated and cleaned to handle missing or outlier values. A time-series alignment is performed to match historical deployment timestamps to their corresponding grid carbon windows. Feature engineering is then applied to structure and normalize this data into a usable feature set for machine learning.
- **Phase 3: Model Training and Prediction (XGBoost):** The structured dataset is fed into an **XGBoost (Extreme Gradient Boosting)** supervised learning model. XGBoost was selected for its high performance with tabular data and its efficiency in handling regression and classification tasks. The model is trained to predict the energy footprint of an incoming deployment based on its estimated duration and current system loads. During a live pipeline run, the AI Decision Engine uses these trained weights to evaluate forecasted carbon data, outputting the optimal timestamp to trigger the deployment via REST API, successfully shifting the workload to a low-carbon window.

# 4. HIGH-LEVEL SYSTEM ARCHITECTURE

## 4.1. System Overview

The proposed sustainable CI/CD release management framework introduces an intelligent decision layer into a conventional CI/CD pipeline (such as Jenkins or GitHub Actions). Instead of executing a static sequence of steps for every code change, this architecture dynamically evaluates the scope of changes, the risk level of the release, and external grid carbon data. By intercepting the standard workflow, the decision layer determines exactly which build, test, and deployment tasks are strictly necessary, thereby bypassing redundant computational workloads and optimizing energy efficiency.

## 4.2. System Components and interaction

The architecture is structured around five core functional components (as defined in the methodology) and an evaluation monitor, all operating within the CI/CD orchestration environment:

1. **Change Impact Analyzer:** This component serves as the entry point of the decision layer. Triggered by a version control commit, it analyzes modified files and evaluates project dependency relationships to map the changes to specific system modules. It outputs a precise list of modules that require rebuilding [10, 5].
2. **Artifact Reuse Engine:** Working in tandem with the Change Impact Analyzer, this engine communicates with an external artifact repository (e.g., Nexus). It applies the decision rule: *IF a valid compiled artifact exists AND the module is unchanged, THEN reuse the artifact; ELSE flag for rebuild*. This prevents the redundant compilation of unmodified component.
3. **Incremental Build Module:** This module executes the compilation phase. Receiving instructions from the Artifact Reuse Engine, it bypasses unchanged modules (using the fetched artifacts) and initiates compilation solely for the affected modules.
4. **Selective Test Execution Engine:** This component governs the release verification stage using a simplified risk-based testing algorithm. It intercepts the pipeline before the testing phase and evaluates the release type. Based on predefined rules, it dynamically triggers the appropriate test categories:
    - *Patch Release:* Executes only **Critical tests** (validating core system functionality).
    - *Minor Release:* Executes **Critical + Integration tests** (verifying component interactions).
    - *Major Release:* Executes the **Full test suite** (including Extended tests for comprehensive validation).
5. **Carbon-Aware Scheduling Layer (Conceptual):** For non-urgent deployment or release tasks, this conceptual layer evaluates external electricity grid carbon-intensity data. It identifies optimal scheduling windows and theoretically delays pipeline execution to align with periods of high renewable energy availability [9], minimizing the carbon footprint of the pipeline.
6. **Evaluation & Telemetry Monitor:** Operating continuously alongside the pipeline, this component satisfies Objective 5 by collecting runtime data. It tracks the pipeline execution time, counts the number of rebuilt modules, measures CPU and

computational workload, and logs estimated energy consumption to allow comparison against the baseline pipeline.

## 4.3. Data and Execution Flow

The operational flow of the architecture proceeds as follows:

- **Step 1 (Trigger & Analyze):** A developer pushes a new commit. The pipeline triggers, and the **Change Impact Analyzer** parses the Git diff to identify modified files and map them to their respective system modules.
- **Step 2 (Evaluate & Fetch):** The **Artifact Reuse Engine** checks the artifact repository. It downloads pre-compiled binaries for any modules not flagged by the analyzer.
- **Step 3 (Build):** The **Incremental Build Module** compiles only the flagged, modified modules, assembling the final build alongside the reused artifacts.
- **Step 4 (Verify):** The **Selective Test Execution Engine** reads the release configuration (Patch/Minor/Major) and executes only the required subset of tests, successfully reducing testing overhead.
- **Step 5 (Schedule):** If the release is non-urgent, the **Carbon-Aware Scheduling Layer** queries carbon intensity APIs and schedules the final deployment during a low-carbon window.
- **Step 6 (Output):** The pipeline completes the release, while the **Evaluation Monitor** outputs the final performance and sustainability metrics for analysis.

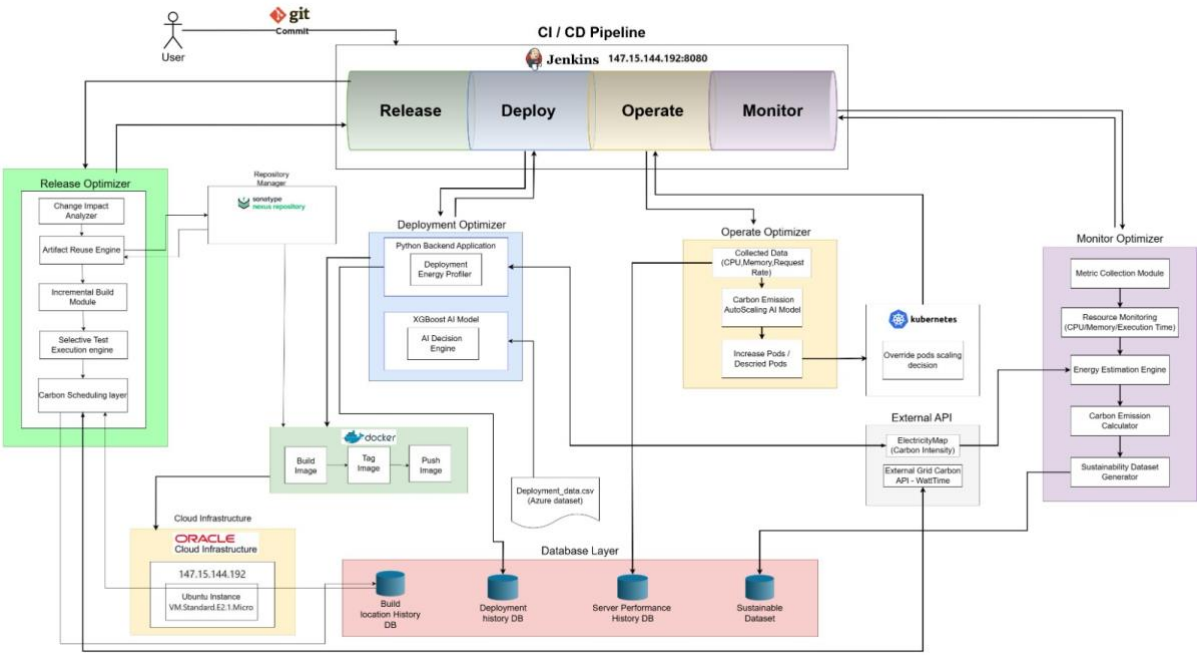



*Figure 1 Overall System Diagram*

## *System Diagram of Components*

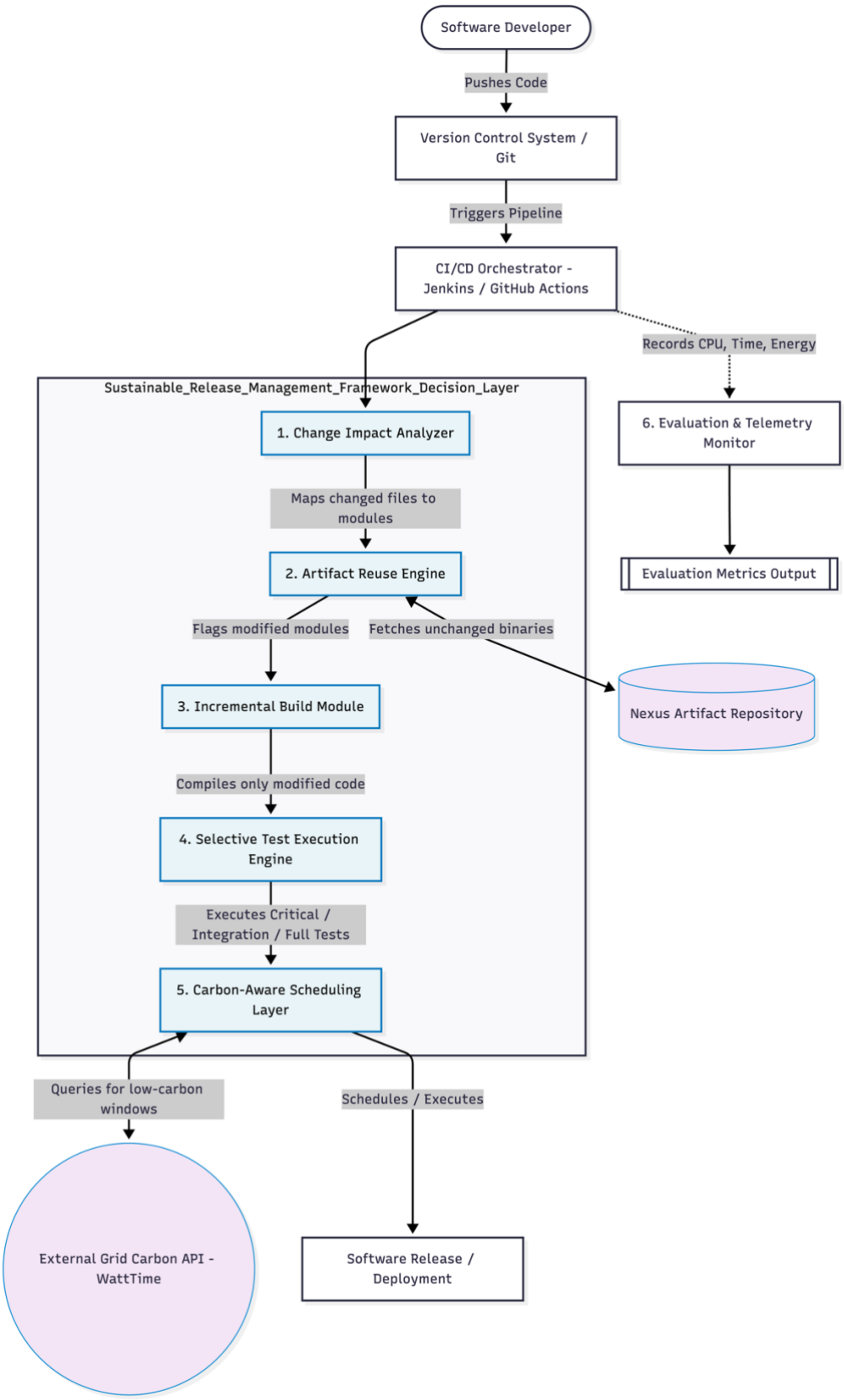



*Figure 2 System Diagram of components*

# 5. User Requirements

## 5.1. Stakeholder Identification

- **Software Developers:** Require rapid feedback loops from the CI/CD pipeline to maintain productivity. They need the system to bypass redundant builds and tests so their code is verified and integrated as quickly as possible.
- **Release/DevOps Engineers:** Require an automated, reliable orchestration layer that manages complex multi-module builds seamlessly. They need the system to reduce infrastructure bottlenecks and optimize CI/CD resource utilization without sacrificing software quality.
- **Sustainability and IT Management:** Require visibility into the environmental impact of software development. They need the system to actively reduce redundant computational workloads and provide metrics on estimated energy savings and carbon footprint reductions.

## 5.2. Functional Requirements

The functional requirements are directly mapped to the core components of the proposed sustainable release management framework:

- **FR1 (Change Impact Analysis):** The system must analyze version control (Git) commits, identify modified files, and map these files to their corresponding dependent modules to determine exactly which components require a rebuild.
- **FR2 (Artifact Reuse):** The system must interface with an artifact repository (e.g., Nexus). It must evaluate the rule: *IF a valid artifact exists AND the module is unchanged, THEN fetch and reuse the artifact*.
- **FR3 (Incremental Compilation):** The system must initiate the build process *only* for the affected modules flagged by the Change Impact Analyzer, successfully bypassing the compilation phase for unmodified modules.
- **FR4 (Selective Test Execution):** The system must classify test cases (Critical, Integration, Extended) and dynamically execute a specific test suite based on the release type:
    - Patch Release Execute Critical tests only.
    - Minor Release Execute Critical + Integration tests.
    - Major Release Execute the Full test suite.
- **FR5 (Carbon-Aware Scheduling):** For non-urgent release tasks, the system must evaluate external grid carbon intensity data to identify and simulate the scheduling of tasks during optimal, low-carbon windows.

- **FR6 (Telemetry and Evaluation Logging):** The system must continuously monitor and record runtime evaluation metrics, specifically: pipeline execution time, the number of rebuilt modules, CPU/computational workload, and estimated energy consumption.

## 5.3. Non-Functional Requirements

- **NFR1 (Performance & Efficiency):** The decision layer must execute its analysis (change detection, artifact fetching, test selection) rapidly. The overhead introduced by the framework must be significantly lower than the time and compute resources saved by skipping redundant builds and tests.
- **NFR2 (Reliability & Correctness):** Artifact reuse and incremental builds must not compromise the integrity of the final software release. The optimized pipeline must produce a functionally equivalent and reliable software build compared to a traditional full-system rebuild.
- **NFR3 (Scalability):** The framework's algorithms must scale efficiently. It must be capable of handling large, complex multi-module projects and extensive test suites without failing or causing pipeline timeouts.
- **NFR4 (Sustainability):** The framework must demonstrably lower the energy consumption and carbon footprint of the CI/CD pipeline, fulfilling its primary environmental objective.
- **NFR5 (Integration & Usability):** The framework must integrate smoothly into modern CI/CD environments (such as Jenkins or GitHub Actions utilizing Docker) without requiring developers to drastically alter their daily coding or version control workflows.

# 6. Commercialization plan

## 6.1. Target Market and Customer Persona

The primary target market for this framework includes medium-to-large software enterprises, digital-native companies, and organizations with high-frequency CI/CD pipelines. The ideal customer personas are DevOps Engineers, Release Managers, and Chief Technology Officers (CTOs) who are looking to optimize cloud infrastructure costs. A secondary market includes organizations with strict Environmental, Social, and Governance (ESG) targets that need to report and reduce their ICT carbon footprint.

## 6.2. Value Proposition

The proposed sustainable release management framework offers a dual-value proposition:

**Economic & Performance Value:** By utilizing artifact reuse and selective test execution, the framework drastically reduces redundant CI/CD compute minutes, leading to lower cloud infrastructure bills and faster feedback loops for developers.

**Sustainability Value:** The framework actively reduces the carbon emissions of software delivery by preventing unnecessary compute cycles and scheduling non-urgent builds during periods of low grid carbon intensity.

### 6.3. Revenue Model and Pricing Strategy

The commercialized product will utilize a B2B Software-as-a-Service (SaaS) tiered subscription model. The pricing strategy is value-based, priced at a fraction of the estimated cloud computing costs the tool saves the enterprise, ensuring a guaranteed positive Return on Investment (ROI).

**Proposed SaaS Revenue Model and Pricing Strategy**

*Table 1 Proposed SaaS Revenue Model and Pricing Strategy*

| Subscription Tier | Target Audience | Estimated Pricing (LKR) | Included Features |
|---|---|---|---|
| **Open-Source / Free** | Individual developers, small open-source projects | LKR 0 | Basic artifact reuse, standard rule-based test selection. |
| **Pro / Team** | Small to medium software development teams | LKR 4,500 / developer / month | Advanced change impact analysis, conceptual carbon scheduling, email support. |
| **Enterprise** | Large organizations and corporate enterprises | Custom (Starting at LKR 150,000 / month) | Self-hosted/on-premise deployment, advanced ESG carbon reporting dashboards, dedicated 24/7 support. |

## 6.4. Competitive Advantage

While major CI/CD providers (like GitHub Actions or Jenkins) offer basic caching mechanisms, they rely on developers to manually configure and optimize them, which is often misconfigured or ignored. This framework provides an automated, intelligent decision layer that handles dependency-aware incremental builds and selective testing out-of-the-box. Native carbon-aware scheduling is also a highly novel feature currently missing from mainstream CI/CD orchestrators.

**6.5 Intellectual Property Considerations** The specific algorithms developed for dynamic change impact analysis and rule-based test selection can be protected as trade secrets or through software patents. The core orchestrator plugin will be proprietary, while adapters for third-party tools (like specific Jenkins plugins) may be released under open-source licenses to encourage integrations.

# 7. Budget and Justification

Since this is a undergraduate research project, The budget relies heavily on existing student resources, open-source software, and cloud-provider free tiers. A formal estimation of the project's resource value and actual out-of-pocket expenses is detailed below.

### Estimated Project Budget and Cost Justification

*Table 2 Estimated Project Budget and Cost Justification*

| Resource Category | Item Description | Estimated Cost (LKR) | Justification |
|---|---|---|---|
| **Hardware** | Development Workstation | LKR 0 | Existing personal resource used for coding and running local Docker containers. |
| **Software** | Git, Apache Maven, Docker, Python, Jenkins | LKR 0 | Utilizing open-source software and free tiers. |
| **Software** | IDE (IntelliJ IDEA Ultimate or VS Code) | LKR 0 | Free via GitHub Student Developer Pack. |
| **Cloud Services** | **Oracle Cloud Infrastructure (OCI)** (Compute & DB) | LKR 0 | Strictly necessary to host the artifact repository and simulate a realistic, distributed CI/CD pipeline for the experimental evaluation phase. |

| | | | |
|---|---|---|---|
| **APIs** | External Grid Carbon API (e.g., WattTime) | LKR 0 | Accessing historical/simulated data using free academic/developer tiers. |
| **Human Effort** | Research & Development Time (480 hours @ LKR 3,000/hr) | *(LKR 1,440,000)* | Commercial equivalent value; provided as an in-kind contribution by the student for academic credit. |
| **Total Expense** | **Actual Out-of-Pocket Budget Required** | **LKR 45,000** | Covers cloud compute provisioning for A/B pipeline testing. |

# WORK BREAKDOWN STRUCTURE (WBS)

### Work Breakdown Structure Table

*Table 3 Work Breakdown Structure*

| WBS ID | Task | Description | Timeline |
|---|---|---|---|
| R2.0 | Project Planning | Initial planning and understanding of sustainable CI/CD release optimization concepts | Feb 2026 |
| R2.1 | Literature Review | Study research on CI/CD optimization, incremental builds, artifact reuse, and carbon-aware scheduling | Feb 2026 |
| R2.2 | Problem Definition | Identify redundant builds, full test execution issues, and sustainability gaps in release pipelines | Feb 2026 |
| R2.3 | Proposal Preparation | Prepare research proposal including methodology and system architecture | Early Mar 2026 |
| R2.4 | Proposal Presentation | Present proposal to evaluation panel | 16th – 18th Mar 2026 |
| R2.5 | System Design | Design architecture of release component (change analysis, build optimization, test optimization, scheduling) | Mar 2026 |
| R2.6 | Change Impact Analyzer Development | Implement Git-based change detection to identify modified modules | Mar – Apr 2026 |
| R2.7 | Dependency Mapping Module | Develop logic to identify dependent modules affected by changes | Mar – Apr 2026 |
| R2.8 | Artifact Reuse Engine | Implement reuse of previously built artifacts from repository (e.g., Nexus) | Apr 2026 |

| | | | |
|---|---|---|---|
| R2.9 | Incremental Build Engine | Develop mechanism to rebuild only affected modules instead of full builds | Apr – May 2026 |
| R3.0 | Test Optimization Engine | Implement selective test execution based on affected modules and risk level | Apr – May 2026 |
| R3.1 | Test Mapping Strategy | Map test cases to modules for efficient test selection | May 2026 |
| R3.2 | Carbon-Aware Scheduling Design | Design scheduling strategy based on carbon intensity data | May 2026 |
| R3.3 | Carbon Data Integration | Integrate carbon intensity APIs (e.g., ElectricityMaps/WattTime) | May – Jun 2026 |
| R3.4 | Scheduling Logic Implementation | Implement logic to schedule builds in low-carbon periods | Jun 2026 |
| R3.5 | CI/CD Integration | Integrate release component with Jenkins / GitHub Actions | Jun – Jul 2026 |
| R3.6 | System Integration | Integrate with monitoring, deploy, and operate components | Jul 2026 |
| R3.7 | Evaluation & Testing | Compare optimized pipeline with traditional pipeline (time, CPU usage, rebuild scope) | Aug 2026 |
| R3.8 | Performance Testing | Evaluate reduction in redundant builds and energy usage | Aug 2026 |
| R3.9 | System Optimization | Improve accuracy and efficiency of optimization techniques | Sep 2026 |
| R4.0 | Documentation & Final Deliverables | Prepare final outputs and documentation | Sep – Oct 2026 |
| R4.1 | Final Report Writing | Complete detailed documentation of release component | Sep – Oct 2026 |
| R4.2 | Website / Demo Preparation | Develop demo and documentation portal | Sep – Oct 2026 |

## 9. GANTT CHART

The following chart visually maps the project milestones across the 9-month development lifecycle (February 2026 to October 2026).

**Project Gantt Chart**

| WBS ID | Task Description | Feb | Mar | Apr | May | Jun | Jul | Aug | Sep | Oct |
|---|---|---|---|---|---|---|---|---|---|---|
| R2.0 | Project Planning | ■ | | | | | | | | |
| R2.1 | Literature Review | ■ | | | | | | | | |
| R2.2 | Problem Definition | ■ | | | | | | | | |
| R2.3 | Proposal Preparation | | ■ | | | | | | | |
| R2.4 | Proposal Presentation | | ■ | | | | | | | |
| R2.5 | System Design | | ■ | | | | | | | |
| R2.6 | Change Impact Analyzer Dev | | ■ | ■ | | | | | | |
| R2.7 | Dependency Mapping | | | ■ | | | | | | |
| R2.8 | Artifact Reuse Engine | | | ■ | | | | | | |
| R2.9 | Incremental Build Engine | | | ■ | ■ | | | | | |
| R3.0 | Test Optimization Engine | | | ■ | ■ | | | | | |
| R3.1 | Test Mapping Strategy | | | | ■ | | | | | |
| R3.2 | Carbon-Aware Scheduling Design | | | | ■ | | | | | |
| R3.3 | Carbon Data Integration | | | | ■ | ■ | | | | |
| R3.4 | Scheduling Logic Implementation | | | | | ■ | | | | |
| R3.5 | CI/CD Integration | | | | | ■ | ■ | | | |
| R3.6 | System Integration | | | | | ■ | ■ | | | |
| R3.7 | Evaluation & Testing | | | | | | | ■ | | |
| R3.8 | Performance Testing | | | | | | | ■ | | |
| R3.9 | System Optimization | | | | | | | | ■ | |
| R4.0 | Final Deliverables | | | | | | | | ■ | ■ |
| R4.1 | Final Report Writing | | | | | | | | ■ | ■ |
| R4.2 | Website / Demo Preparation | | | | | | | | ■ | ■ |

## 10.REFERENCES LIST


[1] M. B. O. A. D. O. P. N. E, "www.multiresearchjournal.com," 25 10 2024. [Online]. Available: https://www.multiresearchjournal.com/admin/uploads/archives/archive-1762931751.pdf. [Accessed 10 03 2026].

[2] C. R. Kasaram, "Spring Boot and Microservices: Engineering Modular and Scalable Back-End Architectures," 01 04 2018. [Online]. Available: https://www.researchgate.net/publication/394920574_Spring_Boot_and_Microservices_Engineering_Modular_and_Scalable_Back-End_Architectures. [Accessed 10 3 2026].

[3] G. Singh, "https://www.eajournals.org/," 17 5 2025. [Online]. Available: https://eajournals.org/ejcsit/vol13-issue23-2025/carbon-aware-resource-allocation-dynamically-balancing-compute-loads-with-renewable-energy-availability/. [Accessed 10 3 2026].

[4] B. S. D. S. V. A. O. D. M. S. T. Z. N. A. J. E. D. G. d. W. B. Peltonen E, "ieeexplore.ieee.org," 2025. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10986765. [Accessed 10 03 2026].

[5] B. D. Curtsinger C, "https://arxiv.org/," 2 9 2021. [Online]. Available: https://arxiv.org/abs/2108.12469. [Accessed 12 2 2026].

[6] M. K. Guillaume Maudoux, "https://ceur-ws.org/," 7 6 2017. [Online]. Available: https://ceur-ws.org/Vol-2070/paper-01.pdf. [Accessed 12 02 2026].

[7] S. G, "eajournals.org," 26 4 2025. [Online]. Available: https://doi.org/10.37745/ejcsit.2013/vol13n23304. [Accessed 10 03 2026].

[8] T. A. C. D. A. Z. Y. Ghaleb, "arxiv.org," 27 01 2026. [Online]. Available: https://arxiv.org/abs/2601.19146. [Accessed 10 03 2026].

[9] N. Reddy Pinnapareddy, "www.researchgate.net," 1 9 2025. [Online]. Available: https://www.researchgate.net/publication/395169347_Carbon_Conscious_Scheduling_in_Kubernetes_to_Cut_Energy_Use_and_Emissions. [Accessed 13 02 2026].

[10] V. Tiinanen, "oulurepo.oulu.fi," 2022. [Online]. Available: https://oulurepo.oulu.fi/bitstream/handle/10024/19703/nbnfioulu-202210133496.pdf;jsessionid=19CBF138F4B822475748E3B1970C05A3?sequence=1. [Accessed 10 03 2026].

[11] M. A. E. Mascheroni, "www.researchgate.net," 2018. [Online]. Available: https://www.researchgate.net/publication/340396275_Continuous_Testing_and_Solutions_for_Testing_Problems_in_Continuous_Delivery_A_Systematic_Literature_Review. [Accessed 10 03 2026].

[12] A. X. A. Aneggi, "arxiv.org," 12 2 2026. [Online]. Available: https://arxiv.org/abs/2602.12081. [Accessed 11 3 2026].

[13] E. e. al. and a. preprint, "The Promise and Reality of Continuous Integration Caching: An Empirical Study of Travis CI Builds," 2026. [Online]. Available: https://arxiv.org/pdf/2601.19146.pdf. [Accessed 10 3 2026].

# 11. APPENDICES

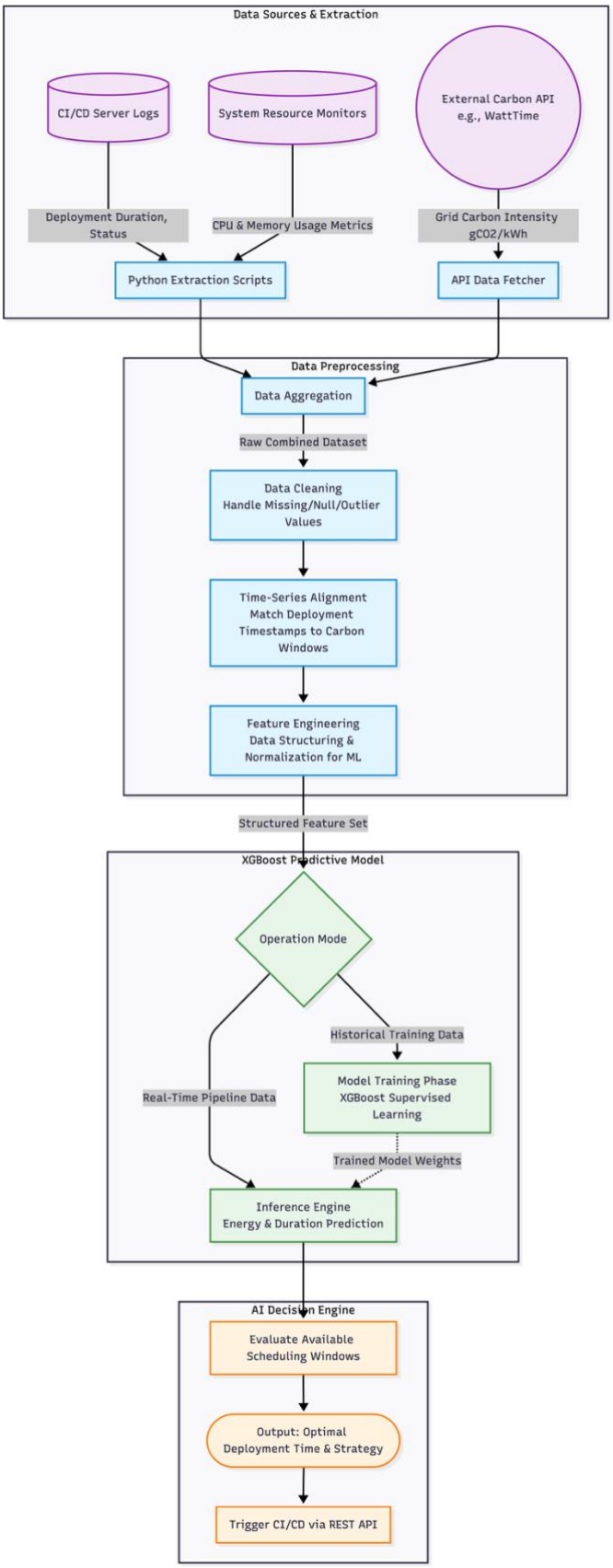
Appendix A: Risk Analysis and Mitigation Plan

The following table outlines the potential risks associated with the development and implementation of the AI-driven sustainable release management framework, along with their assessed impact and planned mitigation strategies.

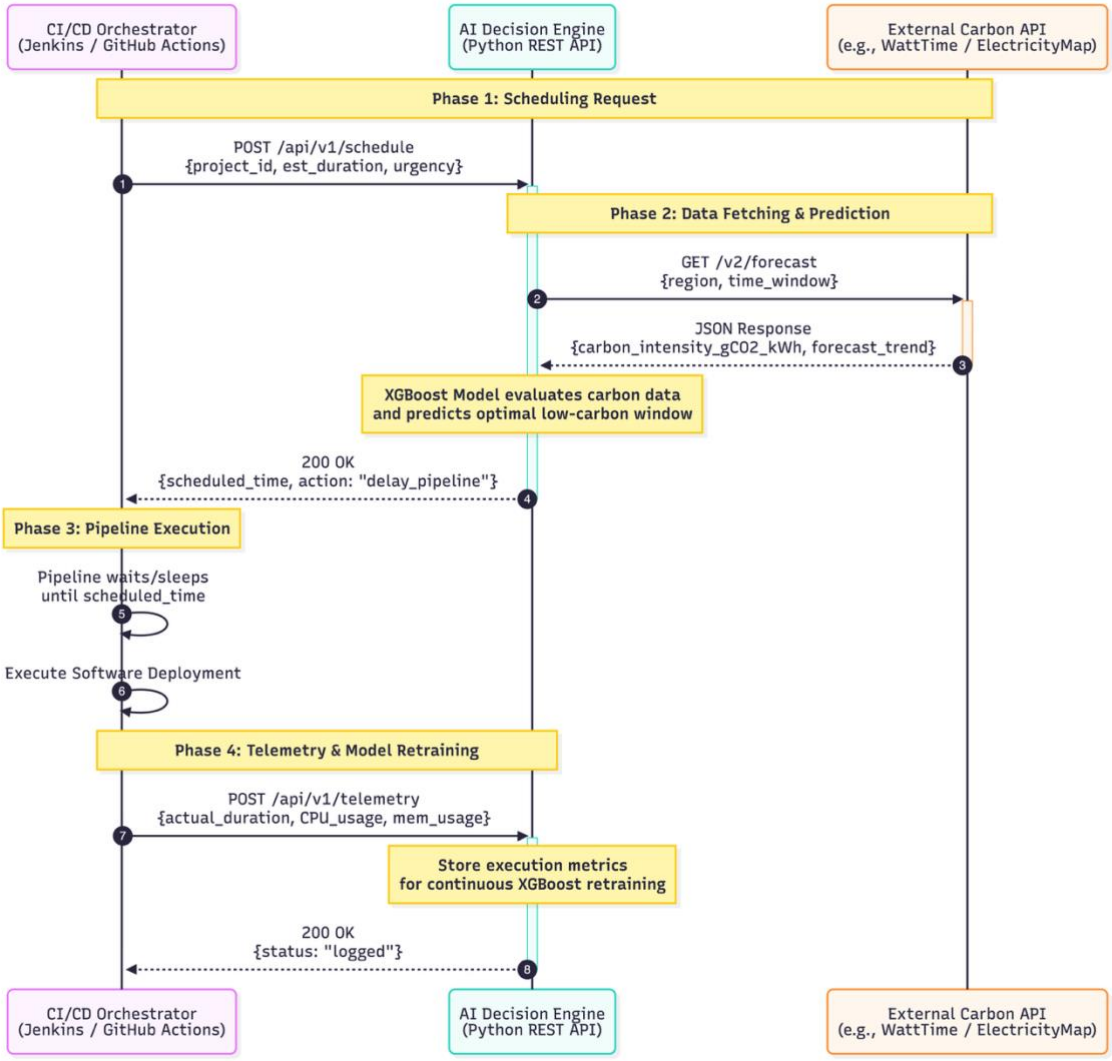
| Risk Category | Risk Description | Probability | Impact | Mitigation Strategy |
|---|---|---|---|---|
| **Data Availability** | Insufficient or poor-quality deployment logs and carbon intensity data required to accurately train the XGBoost predictive model. | Medium | High | Utilize synthetic data generation techniques or leverage existing open-source CI/CD datasets (e.g., TravisTorrent) to supplement the training data if local collection falls short. |
| **External Dependencies** | The external grid carbon intensity APIs (e.g., WattTime or ElectricityMap) become temporarily unavailable, deprecate features, or change their free-tier pricing. | Low | High | Design the system with a fallback mechanism that uses historical, cached carbon data averages or defaults to standard deployment times if the live API is unreachable. |
| **System Integration** | Technical difficulties integrating the custom Python-based REST API (Decision Engine) seamlessly with standard CI/CD tools (Jenkins or GitHub Actions). | Medium | Medium | Conduct extensive local integration testing using Docker containers to simulate the CI/CD environment before attempting cloud deployment. Use well-documented webhook standards. |
| **Performance Overhead** | The Deployment Energy Profiler and AI Decision Engine introduce excessive latency into the CI/CD pipeline, negating the efficiency benefits. | Low | Medium | Keep the XGBoost model lightweight and optimize the REST API response times. Ensure the profiling scripts run asynchronously so they do not block core build tasks. |
| **Schedule / Timeline** | Delays in the Machine Learning model training phase (Phase D3.0) or integration | Medium | High | Adhere strictly to the Work Breakdown Structure (WBS). Allocate buffer weeks |

| | | | | |
|---|---|---|---|---|
| | phase pushing the project past the 6-month deadline. | | | in August and September specifically for troubleshooting and system optimization. |
| **Security** | Vulnerabilities introduced by opening API endpoints for the CI/CD orchestrator to communicate with the Decision Engine. | Low | High | Implement secure authentication tokens for the REST API and restrict access using IP whitelisting to ensure only authorized CI/CD workers can trigger predictions. |

*Appendice 1Risk Analysis and Mitigation plan*




*Appendice 2 Detailed Data flow diagram*




*Appendice 3 Detailed API Interaction diagram*